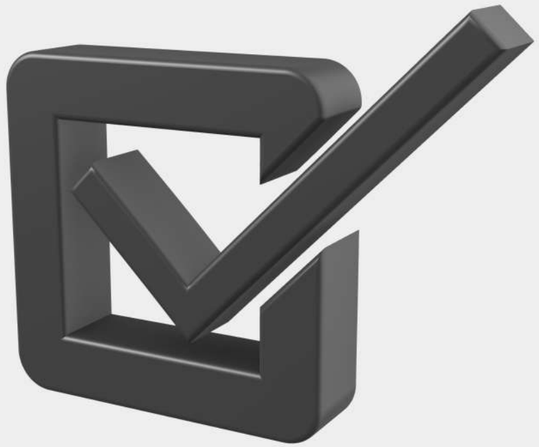
Recommender Systems with Generative Retrieval (TIGER)

Presentado por Leopoldo Farr y Tomás Goas.

ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

Índice - Agenda

- El paper
- Problema
- Nuevo paradigma
- Conceptos relevantes
- Solución
- Referencias



El paper

Presentado en la trigésimo-séptima conferencia anual NeurIPS 2023 en Nueva Orleans, EE.UU.

NEURAL INFORMATION PROCESSING SYSTEMS

Autores

- Shashank Rajput - Universidad de Wisconsin-Madison
- Nikhil Mehta - Google DeepMind
- Anima Singh - Google DeepMind
- Raghunandan H. Keshavan - Google
- Trung Vu, Lukasz Heldt - Google
- Lichan Hong - Google DeepMind
- Yi Tay - Google DeepMind
- Vinh Q. Tran - Google
- Jonah Samost - Google
- Maciej Kula - Google DeepMind
- Ed H. Chi - Google DeepMind
- Maheswaran Sathiamoorthy - Google DeepMind

El problema

Personalización de contenido, recomendar a miles de usuarios

Que el usuario perciba el producto como algo diseñado especialmente para él

El desafío - buscar un nuevo paradigma

Retrieve and rank

Muchos de los principales sistemas modernos lo utilizan para la recomendación secuencial.

Usado por:

- Spotify (Voyager)*
- Google (ScaNN)**
- Entre otros

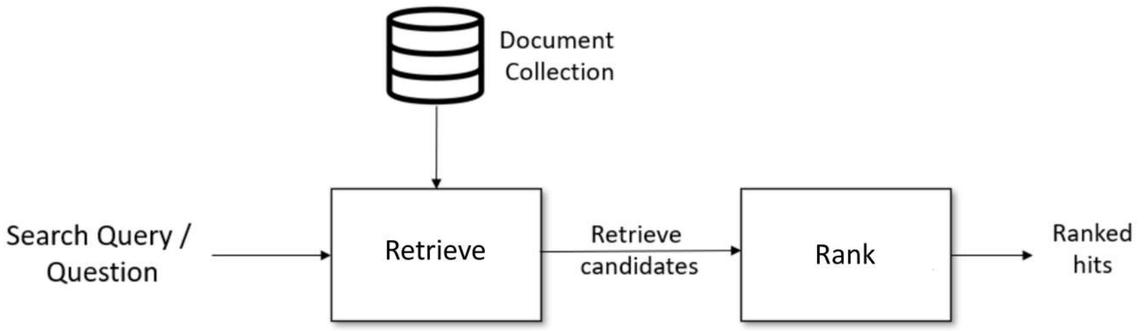


Diagrama simplificado de un sistema Retrieve and Rank

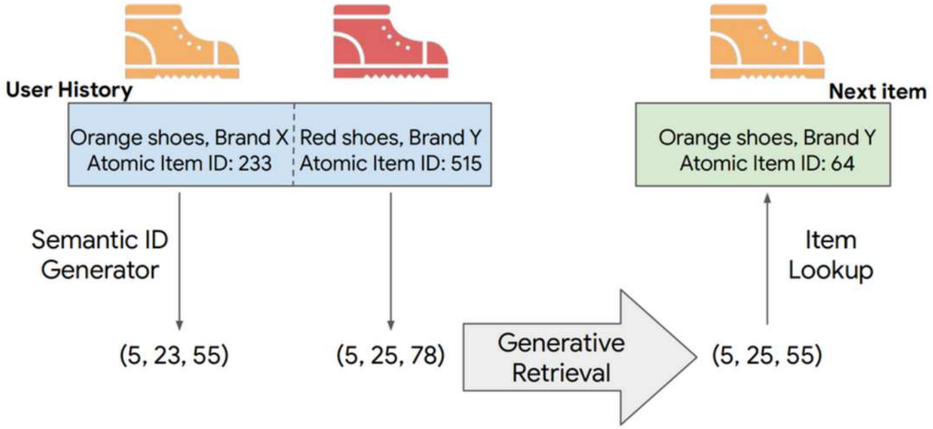
(*) https://engineering.atspotify.com/2023/10/introducing-voyager-spotifys-new-nearest-neighbor-search-library
(**) https://research.google/blog/announcing-scann-efficient-vector-similarity-search

Recomendar de diferente forma

Generative retrieval

¿Y si en vez de buscar los mejores ítems para recomendar, los generamos?

Presentan TIGER (Transformer Index for Generative Recommenders)- Que busca predecir autorregresivamente el "Semantic ID" del siguiente ítem con el que el usuario interactuará.



Antes de entrar de lleno a TIGER

Conceptos relevantes

Buscar que es lo que consumirá el usuario a continuación. Toma en cuenta orden y temporalidad de las interacciones pasadas

Se desmarca de los métodos tradicionales que solo consideran que le gusta al usuario

01

Recomendación secuencial

Transformers

IDs semánticos vs arbitrarios

Antes de entrar de lleno a TIGER

Conceptos relevantes

Modelo de machine learning que busca procesar secuencias de datos. Tomando en cuenta el contexto completo de los datos.

Ideales para recomendación secuencial, pues son capaces de capturar el contexto del historial de interacciones con el ajuste de parámetros adecuado.

02

Recomendación secuencial

Transformers

IDs semánticos vs arbitrarios

Antes de entrar de lleno a TIGER

Conceptos relevantes

Un ID semántico es una representación informativa sobre el contenido del ítem al que está asociado.

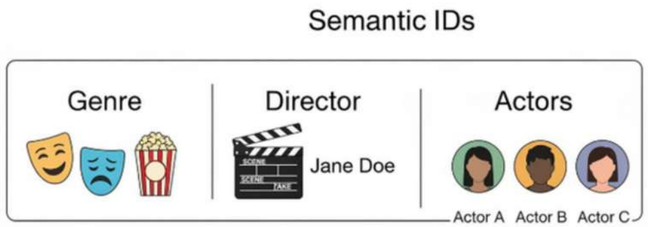
Un ID arbitrario o normal es un indicador único para distinguir un elemento de otro dentro de una base de datos.

Recomendación secuencial

Transformers

03

IDs semánticos vs arbitrarios

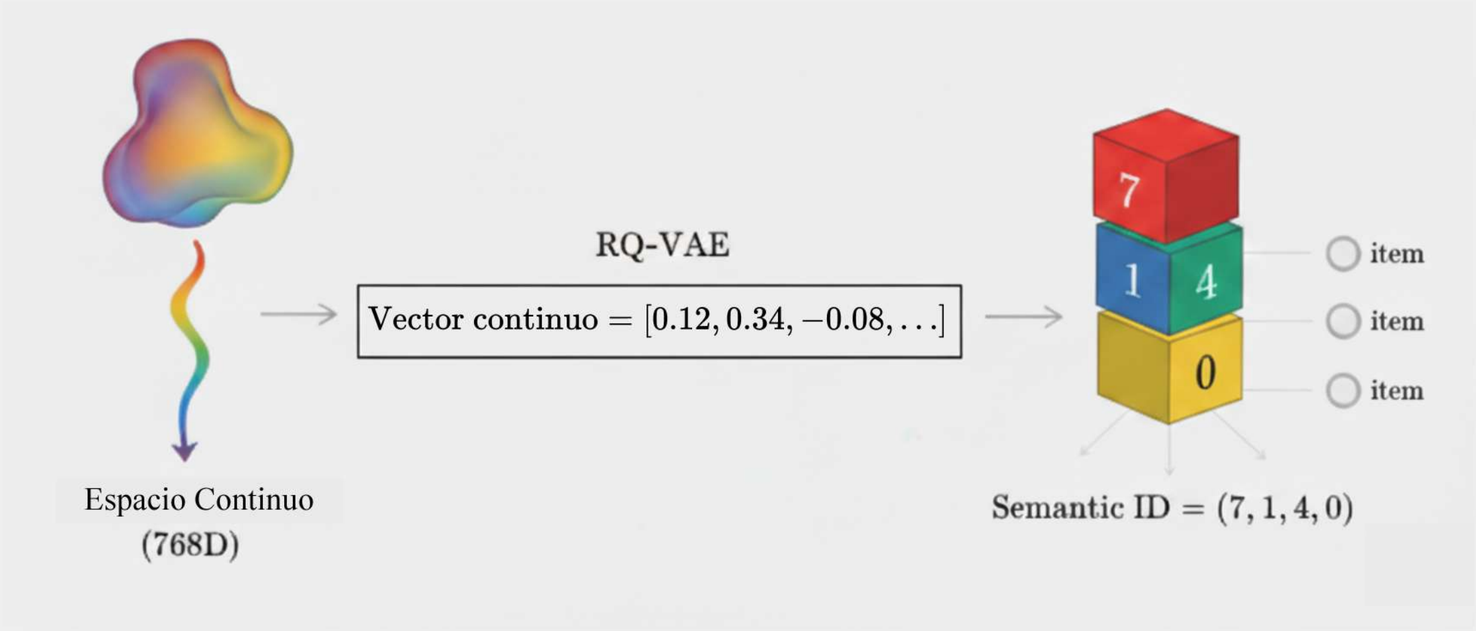


(04, 55, 12)
Género 04
Director 55
Actor 12

Como se generan los IDs Semánticos

Semantic IDs

- Cada ítem se representa inicialmente mediante un embedding continuo de 768 dimensiones.
- Dado que el Transformer requiere entradas discretas similares a los tokens utilizados en texto, el modelo RQ-VAE transforma cada embedding en una tupla corta de números enteros, denominada Semantic ID.
- Los Semantic IDs son semánticos y jerárquicos, de esta forma items similares comparten tokens.

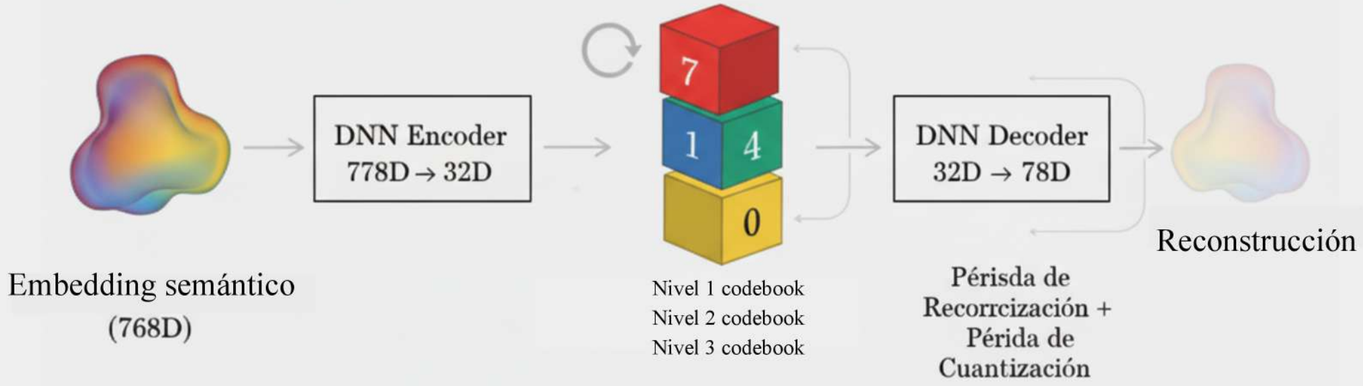


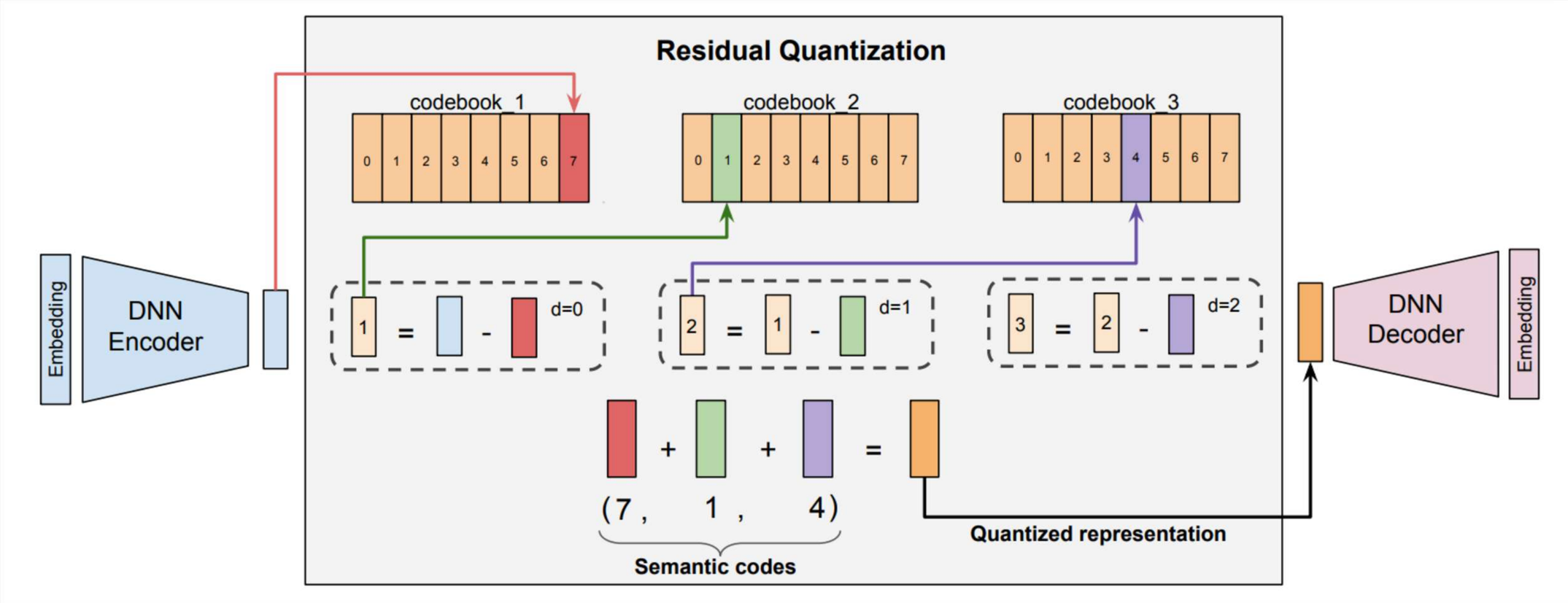
Como se generan los IDs Semánticos

Como de obtiene el Semantic ID con RQ-VAE

Tres componentes:

- **Inicialmente DNN Encoder transforma el embedding semántico de cada ítem (de 768 dimensiones) en una representación latente de 32 dimensiones.**
- **Luego, esta representación se somete a un proceso de cuantización residual jerárquica con tres niveles (m = 3), donde cada nivel dispone de un codebook con 256 vectores de dimensión 32.**
- **Finalmente, un DNN Decoder reconstruye el embedding original a partir del vector cuantizado, lo que permite entrenar el modelo mediante la combinación de una pérdida de reconstrucción y una pérdida de cuantización.**




Residual Quantization
codebook_1
0 1 2 3 4 5 6 7
codebook_2
0 1 2 3 4 5 6 7
codebook_3
0 1 2 3 4 5 6 7
Embedding
DNN Encoder
1 = - d=0
2 = 1 - d=1
3 = 2 - d=2
+ + =
(7, 1, 4)
Semantic codes
Quantized representation
DNN Decoder
Embedding

Entrenamiento del RQ-VAE

$$L(x) = L_{\text{recon}} + L_{\text{rqvae}}$$

$$L_{\text{recon}} = \|x - \hat{x}\|^2$$

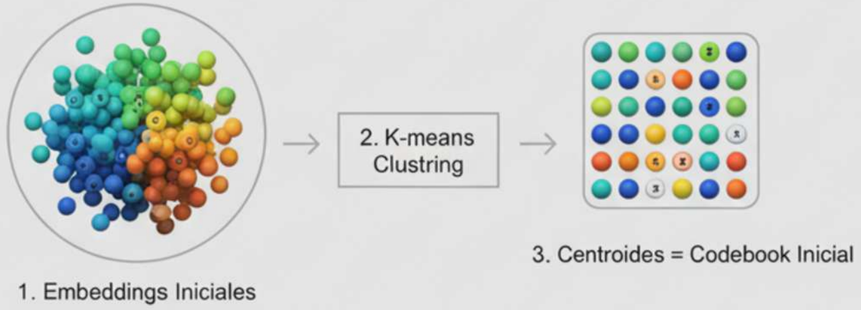
$$L_{\text{rqvae}} = \sum_{d=0}^{m-1} \|\text{sg}[r_d] - c_{d,\ell}\|^2 + \beta \|r_d - \text{sg}[c_{d,\ell}]\|^2$$

- sg (stop-gradiente) evita que los gradientes fluyan por los indices discretos
- β=0.25 controla cuánto se ajustan los codebooks.
- Se entrena todo junto por 20k epochs, hasta lograr ≥80% uso del codebook.

Evitar el codebook collapse: inicialización con k-means

- Durante el entrenamiento, el modelo puede colapsar los codebooks, asignando la mayoría de los embeddings a unos pocos vectores representativos.
- Esto reduce la diversidad y capacidad de representación del espacio latente.
- Se inicializa cada codebook mediante k-means, tomando los centroides obtenidos al aplicar k-means sobre la primera tanda de embeddings del entrenamiento, alcanzando más del 80 % de utilización

- **Solución:**



Colisiones en los Semantic IDs

A veces dos ítems distintos generan el mismo ID semántico (c_0, c_1, c_2).

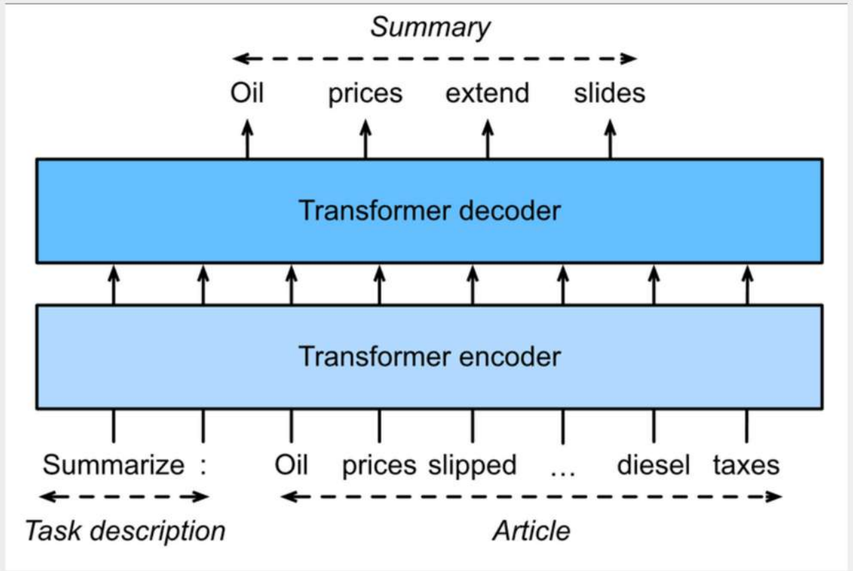
- Solución simple: agregar un cuarto código adicional c_3 que diferencie ambos.
- Si no hay colisión, se asigna igualmente un 0 al final..

Ítem A → (7, 1, 4, 0)

Ítem B → (7, 1, 4, 1)

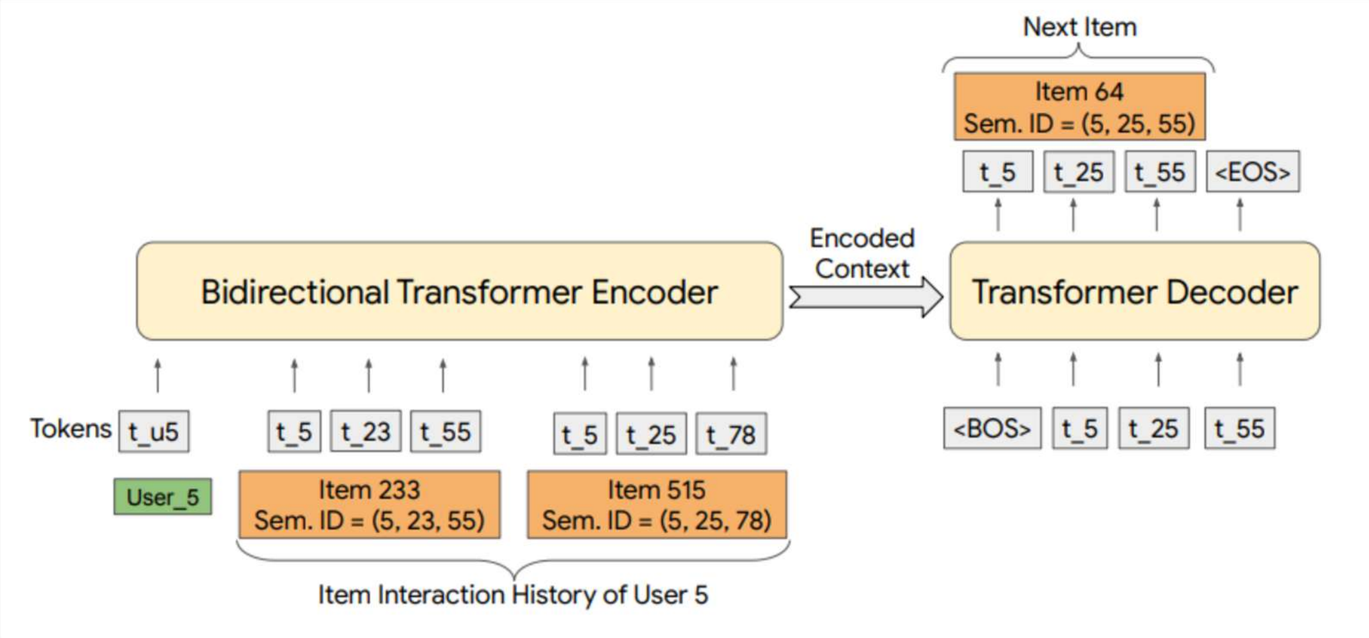
TIGER: un Transformer generativo para recomendación

- El Transformer reemplaza la búsqueda por similitud → ahora genera el próximo Semantic ID token por token.
- Inspirado en modelos de lenguaje generativos tipo T5 de google: input sequence → predicted sequence.



Del RQ-VAE al modelo generativo TIGER

- Ahora que cada ítem tiene un Semantic ID (una secuencia de 4 tokens discretos), podemos tratarlos como palabras en un lenguaje.
- TIGER usa un Transformer encoder–decoder que aprende a generar el próximo Semantic ID dado el historial de interacciones de un usuario.



Cómo el Transformer representa usuarios e ítems

Aspecto	Descripción
Vocabulario	1024 tokens para Semantic IDs (256×4) 2000 tokens para usuarios (hashing trick)
Entrada al modelo	[USER_1] + secuencia de Semantic IDs vistos
Salida esperada	4 tokens del próximo Semantic ID
Arquitectura (T5X)	4 capas encoder + 4 capas decoder 6 cabezas de atención (dim=64) Dropout = 0.1
Tamaño del modelo	~13 millones de parámetros
Entrenamiento	Pérdida de entropía cruzada (token a token, teacher forcing)
Objetivo	Aprender patrones secuenciales y personalización del usuario

Resultados

Se comparó el desempeño general con los metodos sota, las metricas de evaluacion usadas son Recall@K y NDCG@K con K ∈ {5,10}.

Ademas se comparo el metrodo RQ-VAE usado para la generacion de IDs semanticos, con los metodos LSH y Random IDs

Reultados

Comparación General de Desempeño vs SOTA

- Se evaluó TIGER en tres datasets de Amazon Reviews: Beauty, Sports & Outdoors, y Toys & Games.
- Competidores: GRU4Rec, SASRec, BERT4Rec, FDSA, S3-Rec, P5, entre otros.
- TIGER supera consistentemente a todos los métodos SOTA en las tres categorías.

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
P5 [8]	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser [33]	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN [25]	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec [11]	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec [32]	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA [42]	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec [17]	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0463	0.0306	0.0675	0.0374
S^3-Rec [44]	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
TIGER [Ours]	**0.0264**	**0.0181**	**0.0400**	**0.0225**	**0.0454**	**0.0321**	**0.0648**	**0.0384**	**0.0521**	**0.0371**	**0.0712**	**0.0432**
	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

Reultados

Interpretación de los Resultados

TIGER reemplaza el matching de embeddings (usado en modelos tipo dual encoder) por generative retrieval, donde el modelo predice directamente el Semantic ID del próximo ítem.

- En TIGER, cada ítem se representa como un Semantic ID = (c_1, c_2, c_3, c_4), obtenido al cuantizar su embedding de contenido (título, marca, categoría, etc.) con un RQ-VAE.
- Se observó que:
  - El primer token (c_1) agrupa ítems por categoría general (e.g., “Hair”, “Makeup”).
  - Los siguientes tokens (c_2, c_3) refinan subcategorías más específicas.

En resumen, el modelo aprende una representación más rica del espacio semántico de los ítems, lo que se refleja directamente en Recall y NDCG.

Reultados

Comparación de Métodos para Generar Semantic IDs

Se comparó distintos metodos para la generacion de los semantic IDs

- Random ID: Asigna códigos de forma completamente aleatoria, sin considerar la información del ítem, por lo que no preserva la similitud semántica entre ellos..
- LSH (Locality Sensitive Hashing)Proyecta los embeddings sobre hiperplanos aleatorios para generar códigos binarios; los ítems similares tienden a compartir hashes parecidos, aunque el mapeo es rígido y de naturaleza lineal.
- RQ-VAE: usa un autoencoder con residual quantization que aprende no linealmente una representación jerárquica del espacio preservando relaciones semánticas entre los items.

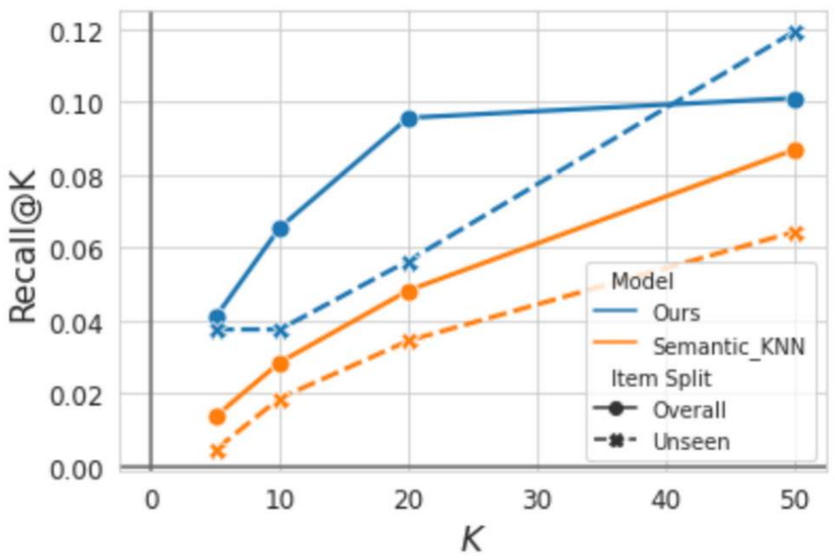
Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
Random ID	0.007	0.005	0.0116	0.0063	0.0296	0.0205	0.0434	0.0250	0.0362	0.0270	0.0448	0.0298
LSH SID	0.0215	0.0146	0.0321	0.0180	0.0379	0.0259	0.0533	0.0309	0.0412	0.0299	0.0566	0.0349
RQ-VAE SID	**0.0264**	**0.0181**	**0.0400**	**0.0225**	**0.0454**	**0.0321**	**0.0648**	**0.0384**	**0.0521**	**0.0371**	**0.0712**	**0.0432**

Generalización a ítems nuevos: el caso Cold-Start

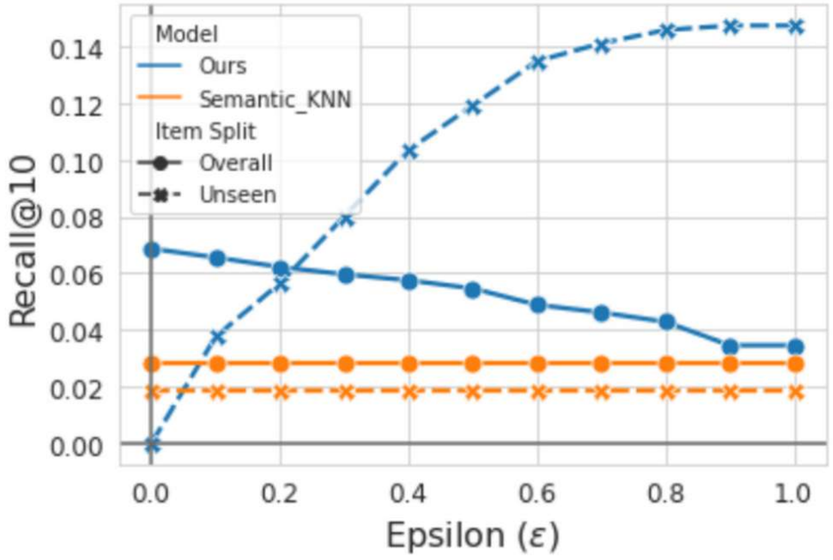
Cold start ocurre cuando el sistema debe recomendar ítems sin información, por ejemplo, con productos recién añadidos a una plataforma.
TIGER se comportó bien.

- Se simuló un escenario donde 5% de los ítems de test no aparecen en el entrenamiento (ítems “unseen”).
- TIGER puede recomendarlos gracias a que sus Semantic IDs se basan en contenido (no en historial).
- Se compara con baseline Semantic_KNN:
- TIGER obtiene mayor Recall@K para todos los valores de K.
- Mejora incluso con $\epsilon \geq 0.1$ (porcentaje de ítems nuevos en top-K).

Generalización a ítems nuevos: el caso Cold-Start



(a) Recall@K vs. K, ($\epsilon = 0.1$).



(b) Recall@10 vs. ϵ.

Figure 5: Performance in the cold-start retrieval setting.

Control de diversidad

En el paper se buscó evitar que el sistema recomiende únicamente ítems muy similares, promoviendo variedad sin sacrificar relevancia.

Para lograrlo, TIGER permite modular la diversidad ajustando la temperatura durante el muestreo autoregresivo del modelo.

- Temperaturas más altas generan una mayor entropía en las recomendaciones, incluyendo categorías complementarias o menos frecuentes.
- Este enfoque ofrece un control explícito entre precisión y diversidad, algo que los modelos tradicionales no permiten de forma directa.

Control de diversidad

Temperature	Entropy@10	Entropy@20	Entropy@50
T = 1.0	0.76	1.14	1.70
T = 1.5	1.14	1.52	2.06
T = 2.0	1.38	1.76	2.28

Target Category	Most-common Categories for top-10 predicted items	
	T = 1.0	T = 2.0
Hair Styling Products	Hair Styling Products	Hair Styling Products, Hair Styling Tools, Skin Face
Tools Nail	Tools Nail	Tools Nail, Makeup Nails
Makeup Nails	Makeup Nails	Makeup Nails, Skin Hands & Nails, Tools Nail
Skin Eyes	Skin Eyes	Hair Relaxers, Skin Face, Hair Styling Products, Skin Eyes
Makeup Face	Tools Makeup Brushes,Makeup Face	Tools Makeup Brushes, Makeup Face,Skin Face, Makeup Sets, Hair Styling Tools
Hair Loss Products	Hair Loss Products,Skin Face, Skin Body	Skin Face, Hair Loss Products, Hair Shampoos,Hair & Scalp Treatments, Hair Conditioners

Referencias

1. Rajput, S., Mehta, N., Singh, A., Keshavan, R. H., Vu, T., Heldt, L., Hong, L., Tay, Y., Tran, V. Q., Samost, J., Kula, M., Chi, E. H., & Sathiamoorthy, M. (2023). Recommender Systems with Generative Retrieval. En arXiv [cs.IR]. http://arxiv.org/abs/2305.05065
2. Sobot, P. (2023). Introducing voyager: Spotify's new nearest-neighbor search library. Spotify Engineering. Recuperado el 12 de noviembre de 2025, de https://engineering.atspotify.com/2023/10/introducing-voyager-spotifys-new-nearest-neighbor-search-library
3. Wikipedia. (s.f.). T5 (language model). En Wikipedia, The Free Encyclopedia. Recuperado el 12 de noviembre de 2025, de https://en.wikipedia.org/wiki/T5_(language_model)
4. Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., & Tagliasacchi, M. (2021). SoundStream: An end-to-end neural audio codec. CoRR, abs/2107.03312. https://arxiv.org/abs/2107.03312
5. Van Den Oord, A., Vinyals, O., et al. (2017). Neural discrete representation learning. Advances in Neural Information Processing Systems, 30. https://papers.nips.cc/paper_files/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html

Q&A

Gracias por su atención

Estamos abiertos a responder dudas